A Pure Arithmetic Data Generation

To train and evaluate digit-level probes and error detectors in the pure arithmetic setting, we generate synthetic data using a two-shot prompting setup with an instruction-tuned language model. The prompting setup includes:

System Message:

```
You are a helpful assistant that calculates the sum of two numbers. Always provide your answer in the format «x+y=z» where x is the first number, y is the second number, and z is their sum. Do not provide any additional explanation.
```

Few-shot Examples: Each prompt includes two demonstrations, where the user asks:

```
Calculate the sum of the following two numbers:
First number: {i}
Second number: {j}
```

The expected assistant response is:

```
«i+j=z»
```

A new target question in the same format is then added to create a complete prompt. This setup is used to generate three-digit addition problems. We extract the residual stream activation at the equals sign (=) in the model's output to form input representations for probe training, with supervision targeting a specific digit (e.g., the hundreds digit) of the result.

We use a 2-shot prompt in the GSM8K setting mainly to ensure that the model adheres to the expected output format. To maintain consistency, we presented the results obtained with 2-shot prompting in the pure arithmetic setting as well. However, we additionally replicated our probing experiments for Gemma2-2b-it under 0-shot and 1-shot settings, and observed similarly strong probe performance across all configurations. Specifically, the best-performing error detectors achieved accuracies of 94.6% (0-shot), 95.8% (1-shot), and 95.4% (2-shot).

In the pure arithmetic setting, the accuracy of Gemma-2-2b-it under 0-shot, 1-shot, and 2-shot prompting is 98.3%, 98.5%, and 98.6%, respectively.

B Sampling Strategy

We construct the probing dataset from model outputs on all addition problems (a, b) where $a, b \in [100, 999]$ and $a + b < 1000$.

Samples are first filtered to include a mixture of both correct and incorrect model predictions. Specifically, the incorrect samples are those where the model's output differs from the true sum in the hundreds digit.

All samples are then grouped based on the hundreds digit of the model's predicted output. From each digit class (2-9), we randomly select up to 100 samples, ensuring that both correct and incorrect predictions are represented across digit classes. If a digit class contains fewer than 100 samples, all available samples from that class are included. For Gemma 2 2B IT, this procedure yields exactly 800 examples. The final dataset is then split into training (70%) and testing (30%) sets.

This sampling strategy achieves:

- A balanced distribution over digit classes (in terms of predicted hundreds digits);
- A representative mix of correct and incorrect predictions, supporting both value probing and error detection tasks.

C Error Detection Probes

Here we provide the formulation for the error-detecting probes discussed in §§ 3.3 and 4.3.

MLP Error Detector (Single). The MLP consists of a single hidden layer with ReLU activation and a hidden dimensionality of 512. As in the logistic probe, the output layer produces 2 logits corresponding to correct and incorrect predictions. The prediction about model correctness $\hat{y}_{\mathrm{c}}$ is given by:

$$\hat{y}_{\mathrm{c}} = \arg\max_i \left(\mathbf{W}_2^\top \operatorname{ReLU}\left(\mathbf{W}_1^\top \mathbf{x}_l + \mathbf{b}_1\right) + \mathbf{b}_2 \right),$$

where $\mathbf{W}_1 \in \mathbb{R}^{d_{\mathrm{model}} \times 512}, \mathbf{W}_2 \in \mathbb{R}^{512 \times 2}, \mathbf{b}_1 \in \mathbb{R}^{512}$, and $\mathbf{b}_2 \in \mathbb{R}^2$ are the probe's parameters. This probe is trained using **cross-entropy loss** on binary labels (1 = correct, 0 = incorrect), and optimized using Adam.

Circular Error Detector (Joint). We combine two circular probes by taking angular difference between their outputs and passing it through a sigmoid to get an output probability value. More

formally, let the probe weights be two vectors $\mathbf{w}_1^{(1)}, \mathbf{w}_2^{(1)}, \mathbf{w}_1^{(2)}, \mathbf{w}_2^{(2)} \in \mathbb{R}^{d_{\text{model}}}$. We compute the probe's output $\hat{y}$ as

$$\theta_1 = \text{atan2}\big(\mathbf{w}_1^{(1)\top}\mathbf{x}_l, \mathbf{w}_2^{(1)\top}\mathbf{x}_l\big), \quad (5)$$

$$\theta_2 = \text{atan2}\big(\mathbf{w}_1^{(2)\top}\mathbf{x}_l, \mathbf{w}_2^{(2)\top}\mathbf{x}_l\big), \quad (6)$$

$$\hat{y}_{\text{c}} = \sigma\big(\theta_1 - \theta_2\big). \quad (7)$$

The probe is trained with binary cross-entropy loss and the weights for the two circular probes are optimized jointly.

Separate Training Strategy. In this approach, two separate digit probes (e.g., logistic, MLP, or circular) are trained independently: one probe is trained to predict the model's actual output ($\hat{y}_{\text{model}}$), while the other is trained to predict the ground truth label ($\hat{y}_{\text{GT}}$). At inference, error detection is performed by checking whether the two probes agree:

$$\hat{y}_{\text{c}} = \mathbb{1}\big[\hat{y}_{\text{model}} = \hat{y}_{\text{GT}}\big] \quad (8)$$

This procedure does not require any supervision and can be applied post hoc to existing probes.

D GSM8K Dataset Construction for Controlled Reasoning

To enable consistent probing and self-correction experiments on symbolic arithmetic within chain-of-thought (CoT) reasoning, we construct a controlled subset of GSM8K problems involving only addition.

Step 1: Filtering addition-only problems. We first identify GSM8K questions whose solutions involve only addition operations. This ensures that intermediate steps are structurally aligned with the arithmetic patterns studied in the pure setting.

Step 2: Abstracting numerical structure. We use Claude 3.7 Sonnet in *extended thinking* mode to extract and abstract each problem's numerical content into symbolic variables (e.g., $\mathtt{x}_1$, $\mathtt{x}_2$, ..., $\mathtt{x}_n$), creating reusable symbolic templates. We manually verified 20 examples produced by this pipeline and found the variable abstraction to be accurate in all cases.

Step 3: Sampling concrete values. We sample all variables independently and uniformly from the range $[100, 999]$. This process allows for consistent arithmetic structure while introducing variation across generated problems.

Example instance. One resulting example from our dataset is:

> **Question:** Sarah is planning to do some baking. She buys 771 pounds of rye flour, 611 pounds of whole-wheat bread flour, and 505 pounds of chickpea flour. Sarah already had 758 pounds of whole-wheat pastry flour at home. How many pounds of flour does she now have?
> **Symbolic variables:** $x_1 = 771$, $x_2 = 611$, $x_3 = 505$, $x_4 = 758$
> **Expected reasoning:**
>
> «$771 + 611 = 1382$»
> «$1382 + 505 = 1887$»
> «$1887 + 758 = 2645$»
>
> **Model response:**
>
> «$771 + 611 + 505 = 1987$»
> «$1987 + 758 = 2745$»
>
> **Predicted answer:** 2745 (incorrect)
> **Correct answer:** 2645

Model Performance of Gemma2-2b-it In the structured CoT setting, 83% of intermediate steps are correctly formatted (i.e., of the form «a+b=c»), and among these, 94.1% yield the correct result. The overall per-problem accuracy, accounting for all outputs including formatting errors, is 29.6%.

E Prompting and Sample Selection in GSM8K

For the self-correction experiments described in Section 5, we use a series of structured prompts to intervene when the error detector flags a potentially incorrect step.

Prompt format. Each prompt includes a short corrective message, followed by the previous equation up to the equals sign («`a + b =`). The following messages are used:

- **suspicious:** *"That step looks suspicious. Let's re-do just this step:"*
- **neutral:** *"That step looks incorrect. Let's re-do just this step:"*
- **specific:** *"The calculation in this step is incorrect. Let's recalculate:"*

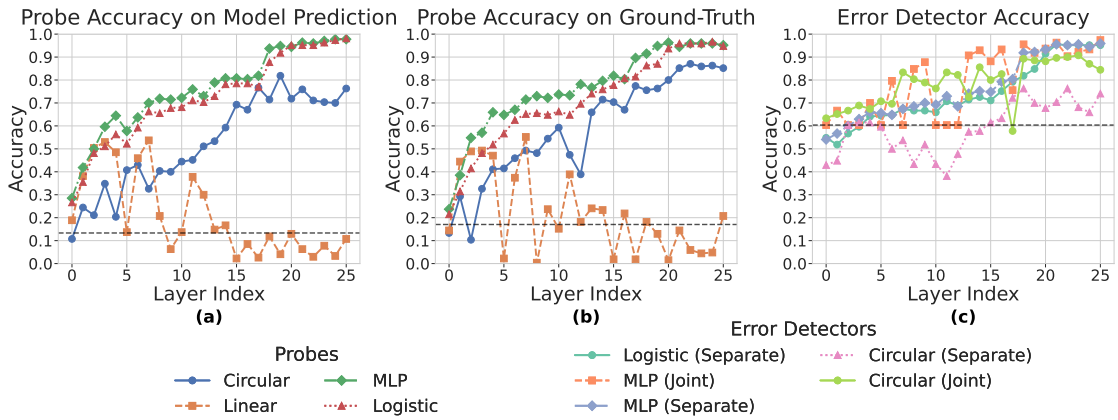

Figure 6: **Probing 3-Digit Arithmetic Queries (Subtraction).** Probing and error detection results on subtraction tasks. As with addition, probes reliably decode both model predictions (a) and ground-truth answers (b), and error detectors achieve high accuracy in predicting correctness (c), confirming the robustness of our findings.

- **stronger:** *"That's definitely wrong. The correct calculation should be:"*
- **detailed:** *"I made an error in adding these numbers. Let me compute the sum correctly step by step:"*

Sample selection. In both the probing and intervention experiments on GSM8K, we select a single step per response for evaluation:

- We first generate the full model response to a given question.
- We identify all equations in the format «`a + b = c`» and determine whether they are correct.
- If all equations are correct, we select the first one and extract the residual stream activation at the equals sign (=) as a **correct** sample.
- If any equation is incorrect, we select the first incorrect one and extract the activation at its equals sign as an **error** sample.

This means we test at most one equation per model response. The same procedure is used both for probe training and evaluation, and for the prompting-based self-correction experiments.

F Error Statistics

We evaluated Gemma 2 2B IT on 319,972 samples of the form (x, y) where $x + y < 1000$. The model produced 3,329 (1.04%) incorrect results. Among these errors: 537 (∼16% of the total errors) had incorrect ones digits, 2,203 (∼66%) had incorrect tens digits. 3,328 (∼100%) had incorrect hundreds digits.

G Computational Resources and Tools

All experiments were conducted on a single Nvidia RTX 4090 GPU. The longest training run for the probes took 1 hour. Generating model response for the experiments in §4 took 90 GPU hours. Our experiments were carried out using `PyTorch` (Paszke et al., 2019) and HuggingFace `transformers` (Wolf et al., 2020). All models were run with `bfloat16` precision for efficient memory usage. For some of our experiments, we used a subset of the GSM8K dataset (Cobbe et al., 2021), which is available through an MIT license.[5] We performed our data analysis using `NumPy` (Harris et al., 2020) and `Pandas` (Wes McKinney, 2010). The paper's bibliography was curated using `Ryanize-bib` (Zouhar, 2023).

H Additional Results

In this section, we extend our main findings by evaluating the generality of our probing methodology across a broader range of settings. In particular, we consider arithmetic tasks involving both addition and subtraction (Appendix H.1), problems with a wider operand range (up to 5-digit numbers; Appendix H.2), and free-form CoT reasoning with unconstrained natural language outputs (Appendix H.3).

H.1 Generalization to Subtraction

We construct a dataset with 800 addition and 800 subtraction queries, randomly mixing them. We train and evaluate digit-level probes and error detectors following the procedure in §3. Results in Fig-

[5] https://choosealicense.com/licenses/mit/